Indian Institute of Technology Gandhinagar
BE623 — Biocomputing | Sem 1, 2026-2027

Lab Session 4: Git, GitHub, Jupyter & Biological Databases

Version Control, Notebooks, and a Tour of NCBI, UniProt, PDB & ENCODE

Learning Objectives

By the end of this session, you will be able to:

- Initialize a Git repository, track changes, and write meaningful commits.
- Push, pull, and clone a repository using GitHub, and explain fork vs. clone.
- Create and run a Jupyter notebook, and distinguish code cells from markdown cells.
- Describe what NCBI, UniProt, PDB, and ENCODE each store and when to use each one.
- Explain what an API is, and construct a basic REST request against NCBI's E-utilities API.

Prerequisites

Before starting, you should be able to:

- Navigate directories and manage files using bash (Lab Session 2).
- Write and run a simple bash script with a shebang line.

Lab Bench Setup

Create your workspace for this session and add two starter files to work with:

```
$ mkdir -p ~/be623_lab4
$ cd ~/be623_lab4
$ echo "# BE623 Lab 4" > README.md
$ echo "Notes for today's session." > notes.txt
$ ls
```

Expected: *README.md and notes.txt are present in ~/be623_lab4.*

File Manifest

Filename	Description	Used In
README.md	Starter file for Git tracking	Exercise 1, 2
notes.txt	Starter file for Git tracking	Exercise 1, 2
.gitignore	Generated: excludes files you don't want tracked	Exercise 1
lab4_notebook.ipynb	Jupyter notebook you build in Exercise 3	Exercise 3, Post-Lab

Before You Start

Confirm both tools are available:

```
$ git --version
$ jupyter --version
```

NOTE: *If either command is not found, ask a TA before continuing. The rest of this session depends on both being installed.*

Exercise 1: Git Basics

1.1 Initializing a Repository

git init creates a hidden .git directory that tracks all history for this folder going forward.

```
$ git init
$ git status
```

Expected: *git status reports an empty repository with untracked files (README.md, notes.txt).*

1.2 Staging, Committing, and Viewing History

Staging (add) and committing are two separate steps. This lets you build a commit from only the changes you intend to save.

```
$ git add README.md notes.txt
$ git commit -m "Initial commit: add README and notes"
$ git log
```

Expected: *git log shows one commit with your message, author, and timestamp.*

NOTE: *A good commit message describes what changed and why. Eg: "Add X", not "Added X" or "Adding X".*

Make a small change and repeat the cycle to see history grow:

```
$ echo "Second line of notes." >> notes.txt
$ git add notes.txt
$ git commit -m "Add a second line to notes"
$ git log --oneline
```

Expected: *git log --oneline now shows two commits, most recent first.*

1.3 Ignoring Files with .gitignore

Not everything belongs in version control. Generated output files and local scratch files should be excluded.

```
$ touch scratch.log
$ echo "scratch.log" > .gitignore
$ git add .gitignore
$ git commit -m "Add gitignore for scratch files"
$ git status
```

Expected: *git status no longer lists scratch.log as untracked, even though the file still exists.*

TIP: *One pattern per line in .gitignore. Use *.log to ignore every file with that extension instead of naming each one.*

Exercise 2: GitHub

2.1 Creating a Remote Repository and Pushing

A remote is just a named URL your local repository knows about. Create an empty repository on github.com first (do not initialize it with a README or .gitignore), then connect it.

```
$ git remote add origin https://github.com/<username>/be623-lab4.git
$ git branch -M main
$ git push -u origin main
```

Expected: *Refreshing the GitHub repository page in your browser shows your committed files.*

2.2 Cloning and Pulling

clone creates a fresh local copy of a remote repository, including its full history. Its useful for working from a second machine or checking a classmate's repository.

```
$ cd ~
$ git clone https://github.com/<username>/be623-lab4.git be623-lab4-copy
$ cd be623-lab4-copy
$ git pull
```

Expected: *be623-lab4-copy contains the same files as your original working directory; git pull reports "Already up to date."*

NOTE: *clone is a one-time copy operation; pull is what you run afterward to fetch new commits into a repository you already have locally.*

2.3 Fork vs. Clone

These solve different problems, and it is a common point of confusion:

	Fork	Clone
What it does	Makes you a personal copy of someone else's repo, on GitHub	Downloads a repo (yours or anyone's) to your local machine
Where it lives	GitHub (server-side)	Your computer (local)
Typical use	Contributing to a project you don't own	Working on any repo you already have access to

Exercise 3: Jupyter Notebooks

3.1 Launching and Cell Types

A notebook is a sequence of cells you run independently, and in any order. Code cells execute code; markdown cells render formatted text.

```
$ jupyter notebook
```

Expected: *A browser tab opens showing the Jupyter file browser; create a new notebook from New > Python 3, and save it as lab4_notebook.ipynb.*

NOTE: *Press Esc then M to convert a cell to markdown, or Esc then Y to convert it back to code.*

3.2 Saving, Checkpoints, and Export

Jupyter autosaves periodically and also lets you create a manual checkpoint you can revert to. Add one markdown cell and one code cell (e.g. print("hello")) to your notebook, then save it.

Expected: *The .ipynb file is present in your working directory, with both a markdown and a code cell visible.*

TIP: *A .ipynb file is JSON internally, which makes its Git diffs noisy. You'll see this in the next step.*

3.3 Committing a Notebook

Track your notebook the same way you tracked README.md and notes.txt earlier:

```
$ git add lab4_notebook.ipynb
$ git commit -m "Add lab 4 notebook"
$ git push
```

Expected: *The notebook appears in your GitHub repository after refreshing the page.*

NOTE: *git diff on a .ipynb file shows raw JSON changes (cell metadata, execution counts), not a clean line-by-line view of your code.*

Exercise 4: Biological Databases — A Tour

These four databases store different views of the same biology. We'll follow one gene: human hemoglobin beta (HBB), through all four, in your browser.

4.1 NCBI — Sequence Records

NCBI hosts nucleotide and protein sequence records (GenBank), each with a unique accession number. Search NCBI Nucleotide for accession NM_000518 and note the organism and sequence length.

Expected: *A GenBank record for the human HBB (hemoglobin subunit beta) mRNA.*

4.2 UniProt — Protein Sequence & Annotation

UniProt focuses on protein sequences plus curated annotation like function, domains, disease links, etc., Search UniProt for accession P68871 and note its function description and length.

Expected: *The UniProt entry for human hemoglobin subunit beta, with a Function section and sequence viewer.*

4.3 PDB — 3D Structures

The Protein Data Bank (RCSB PDB) stores experimentally determined 3D structures. Search RCSB PDB for entry 1HHO and note the experimental method and resolution.

Expected: *The 1HHO structure page: human oxyhemoglobin, solved by X-ray diffraction.*

Exercise 5: Introduction to APIs & REST

So far you've reached every database through a browser, by hand. An API lets a program ask a database for exactly the record it needs, directly. This is how real pipelines pull data at scale instead of clicking through search results one at a time.

5.1 What Is an API?

An API (Application Programming Interface) is a defined way for one piece of software to ask another for data or action, without needing to know how the other side is built internally. A REST API is a common style of API built on top of plain web requests which is the same underlying mechanism your browser uses to load a page, just returning data instead of a rendered page.

NOTE: *If you can open a URL in a browser and read the response, you've already made a REST API call. There's no special tool required to try one.*

5.2 Anatomy of a REST Request

Every REST request is built from the same pieces:

Part	Role	Example
Base URL	The server you're talking to	https://eutils.ncbi.nlm.nih.gov/entrez/eutils/
Endpoint	Which operation you want	efetch.fcgi
Parameters	The specifics of your request	?db=nucleotide&id=NM_000518
Method	The type of request (usually GET for fetching data)	GET
Response format	How the data comes back	XML, JSON, or plain text (FASTA, in NCBI's case)

TIP: *Parameters after the ? are joined with & ; Think of them as named settings you're passing in, similar to flags on a command-line tool.*

5.3 Reading API Documentation

Every well-documented API publishes: (1) its base URL, (2) a list of endpoints and what each does, (3) which parameters each endpoint accepts (and which are required vs. optional), and (4) what the response looks like. When you meet a new API, look for exactly these four things before writing any code. The documentation page is telling you how to fill in the table from 5.2.

5.4 NCBI's REST API — E-utilities

NCBI's E-utilities is a REST API in front of the same records you browsed in Exercise 4.1. Two endpoints are enough to get started: esearch (find IDs matching a search term) and efetch (retrieve a full record by ID).

Paste this URL directly into your browser's address bar:

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi?
db=nucleotide&term=HBB[gene]+AND+human[orgn]
```

Expected: *An XML response containing a list of matching record IDs (a <Count> and several <Id> tags).*

Now fetch one specific record directly by accession, using efetch:

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?
db=nucleotide&id=NM_000518&rettype=fasta&retmode=text
```

Expected: *The plain-text FASTA sequence for NM_000518. Its the same record you looked up by hand in Exercise 4.1, now retrieved with a single URL.*

Change retmode=text to retmode=xml and reload the URL to see the same record returned as structured XML instead of plain FASTA.

Expected: *An XML document with the same sequence data, now wrapped in tags describing each field.*

NOTE: *This is exactly the pattern you'll use from code in a later session: A program builds this same URL, sends the request, and parses whatever comes back. Today's goal is just recognizing the pattern, by hand.*

Troubleshooting Guide

Problem	Cause	Solution
git commit says "nothing to commit"	Changes were never staged with git add	Run git add <file> (or git add .) before git commit
git push is rejected	GitHub repo was initialized with a README, so histories diverge	Recreate the remote without a README, or run git pull --rebase origin main first
.gitignore doesn't seem to work	File was already tracked by Git before being added to .gitignore	Run git rm --cached <file> once, then commit. .gitignore only affects untracked files
Jupyter can't find the kernel / wrong Python version	Notebook launched from a different conda/venv environment	Activate the correct environment before running jupyter notebook
Notebook won't save / "autosave failed"	Browser tab lost connection to the Jupyter server	Check the terminal running jupyter notebook is still active; refresh the browser tab

Problem	Cause	Solution
Can't reach NCBI / UniProt / PDB / ENCODE website	Institute network or proxy blocking the request	Check network connectivity; ask a TA about proxy settings if on institute Wi-Fi
E-utilities URL returns an error or empty response	A required parameter is missing, or db/id value is misspelled	Check the parameter names against Exercise 5.2. db, id, and rettype are case-sensitive

Post-Lab Exercises

Complete these after the session to reinforce your skills:

1. Initialize a new Git repository in a different folder, make at least three separate commits, and push it to a new GitHub repository.

2. Add a .gitignore to that repository that excludes at least one file type you generate often (e.g. *.log or *.tmp).

3. In lab4_notebook.ipynb, add a markdown cell summarizing in your own words what NCBI, UniProt, PDB, and ENCODE each store. Commit and push the updated notebook.

4. Pick any human gene you're curious about and find its record in all four databases. Write down the accession/ID you found in each.

5. Using esearch, find the NCBI nucleotide ID(s) for a gene of your choice, then use efetch to retrieve one of those records as FASTA, by editing the URLs from Exercise 5.4 directly in your browser.

6. Stretch(optional): fork a classmate's be623-lab4 repository on GitHub, clone your fork locally, and open it in Jupyter.

Clean Up

Stop the Jupyter server with Ctrl+C in its terminal, then archive your work directory:

```
$ cd ~
$ tar -czf be623_lab4_backup.tar.gz be623_lab4
```

APPENDIX

Quick Reference

(Detachable reference — Git/GitHub commands and database summary)

Git & GitHub Quick Reference

Command	Purpose
git init	Start tracking the current directory
git status	Show staged / unstaged / untracked files
git add <file>	Stage a file for the next commit
git commit -m "msg"	Save staged changes as a commit
git log / git log --oneline	Show commit history
git remote add origin <url>	Link a local repo to a GitHub remote
git push -u origin main	Upload commits to the remote
git clone <url>	Copy a remote repository locally
git pull	Fetch and merge new commits from the remote

Database Quick Reference

Database	Stores	Typical ID
NCBI	Nucleotide & protein sequence records (GenBank)	NM_000518
UniProt	Curated protein sequence + functional annotation	P68871
PDB (RCSB)	Experimentally determined 3D structures	1HHO

NCBI E-utilities Quick Reference

Endpoint	Purpose	Key Parameters
esearch.fcgi	Search for IDs matching a term	db, term
efetch.fcgi	Retrieve a full record by ID	db, id, rettype, retmode

End of Lab Manual